

JavaScript Array const

[Back to JS page](#)

Table of Contents

- [JavaScript Array const](#)
 - [Cannot be Reassigned](#)
 - [Example 1](#)
 - [Elements Can be Reassigned](#)
 - [Example 2](#)
 - [Const Block Scope](#)
 - [Example 3](#)
 - [Redeclaring Arrays](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

Cannot be Reassigned

Arrays declared with `const` cannot be reassigned to a different array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Reassigned</h4>
<p id="demo"></p>

<script>
const cars = ["Volvo", "BMW", "Toyota"];

// This works - modifying elements is allowed
cars[0] = "Honda";

// But this would cause an error (reassignment)
// cars = ["Audi", "Ford"]; // TypeError: Assignment to constant variable

document.getElementById("demo").innerHTML =
  "const cars: " + cars + "<br><br>" +
  "You can modify elements, but NOT reassign the array";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Elements Can be Reassigned

While the array itself cannot be reassigned, its elements CAN be changed. This is because `const` only prevents reassignment of the variable, not modification of the array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Elements Can be Reassigned</h4>
<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange"];

// Element reassignment is allowed
fruits[0] = "Mango";
fruits[1] = "Apple";

// push() and pop() are also allowed
fruits.push("Kiwi");
fruits.pop();

// Const only protects the variable, not the array contents
document.getElementById("demo").innerHTML =
  "fruits after changes: " + fruits;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Const Block Scope

Arrays declared with `const` have block scope. They are not accessible outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Const Block Scope</h4>
<p id="demo"></p>

<script>
{
  const cars = ["Volvo", "BMW"];
  document.getElementById("demo").innerHTML +=
    "Inside block: " + cars + "<br>";
}

// cars is NOT accessible here
document.getElementById("demo").innerHTML +=
  "Outside block: cars is not accessible (block scope)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Redeclaring Arrays

A `const` array cannot be redeclared in the same scope. It can be redeclared in a different block scope:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Array const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring Arrays</h4>
<p id="demo"></p>

<script>
const cars = ["Volvo", "BMW"];

// This is OK - different block scope
{
  const cars = ["Toyota", "Honda"];
  document.getElementById("demo").innerHTML +=
    "Inside block: " + cars + "<br>";
}

// This is OK - different block scope again
{
  const cars = ["Audi", "Ford"];
  document.getElementById("demo").innerHTML +=
    "Second block: " + cars + "<br>";
}

// cars still refers to the original array
document.getElementById("demo").innerHTML +=
  "Outer scope: " + cars;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Array const](#)